

CryptoCast

Complete Project Guide

From Setup to Server — Everything You Need to Know

Multi-Horizon Bitcoin Price Forecasting Using Deep Learning
GUVI × HCL Master Data Science Program — Capstone Project

Table of Contents

- 1. Project Overview
- 2. Setup & Installation
- 3. Pipeline Execution (Step-by-Step)
- 4. Starting the Streamlit Server
- 5. Step 1: Data Loading, Cleaning & EDA
- 6. Step 2: Model Training (PyTorch)
- 7. Step 3: Visualization Generation
- 8. Step 4: Walk-Forward Validation
- 9. Model Architectures Deep Dive
- 10. Dashboard Pages (10 Pages)
- 11. Key Data Science Concepts for Viva
- 12. Results Summary
- 13. Common Viva Questions & Answers

1. Project Overview

CryptoCast is a multi-horizon Bitcoin price forecasting system that compares 5 deep learning architectures to predict BTC closing prices at 3 horizons — **1-day**, **3-day**, and **7-day** ahead — using a 60-day lookback window of 9 features.

Model	Type	Key Strength
1D-CNN	Convolutional	Fast local pattern extraction
RNN	Recurrent (Simple)	Baseline sequential model
LSTM	Recurrent (Gated)	Long-term memory retention
Transformer	Attention-based	Global context via self-attention
PatchTST	Patch + Transformer + RevIN	State-of-the-art, best overall

2. Setup & Installation

Step 1: Clone the Repository

```
git clone https://github.com/pravinkumardatafreak/cryptocast.git
cd cryptocast
```

Step 2: Create Virtual Environment

```
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # macOS/Linux
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Key Dependencies:

Package	Purpose
torch	PyTorch deep learning framework
pandas, numpy	Data manipulation
scikit-learn	MinMaxScaler, metrics (MAE, RMSE, MAPE)
matplotlib, seaborn	Static EDA visualizations
streamlit	Interactive dashboard
plotly	Interactive charts in dashboard
groq	LLM integration for AI insights

Step 4: Prepare Data

Place your Bitcoin data CSV in data/ as Bitcoin Historical Data.csv (Investing.com format) or btc_data.csv (pre-cleaned).

Required columns: Date, Price, Open, High, Low, Vol., Change %

3. Pipeline Execution

Running the Full Pipeline:

```
bun run pipeline  
# OR directly:  
python cryptocast.py
```

This single command runs ALL 4 steps in sequence:

Step	What It Does
Step 1 (EDA)	Load CSV, clean data, generate 8 EDA plots, scale features
Step 2 (Training)	Train 5 models (1D-CNN, RNN, LSTM, Transformer, PatchTST)
Step 3 (Visualization)	Generate comparison charts (loss curves, actual vs predicted)
Step 4 (WfV)	Walk-Forward Validation backtesting (2 halving-aligned folds)

Individual Step Commands:

Command	What It Does
bun run eda	Run Step 1 only (EDA & preprocessing)
bun run train	Run Step 2 only (model training)
bun run wfv	Run Step 4 only (backtesting)

4. Starting the Streamlit Server

Launch Command:

```
bun run dev  
# OR directly:  
streamlit run app.py --server.port 8502
```

What happens when you run this:

1. Streamlit starts a local web server
2. Opens your browser at <http://localhost:8502>
3. The landing page (app.py) loads with project overview
4. The sidebar shows navigation to all 10 pages
5. Pages are auto-discovered from the pages/ directory

Important: The pipeline (Steps 1-4) must be run *BEFORE* launching the dashboard, because the dashboard reads pre-computed results (JSON files, visualizations, model weights).

5. Step 1: Data Loading, Cleaning & EDA

File: src/step1_eda.py

5.1 Data Loading & Cleaning

Loads from `Bitcoin Historical Data.csv` (Investing.com format). Handles string-to-float conversions for prices with commas. Parses volume abbreviations: 2.5K → 2500, 1.2M → 1,200,000, 3.1B → 3,100,000,000. Output: ~4,964 rows × 6 columns.

5.2 EDA Visualizations

Generates 8 static plots saved to `visualizations/`:

#	Plot	Purpose
01	Price Time Series	Shows BTC price trajectory over 14 years
02	Volume Plot	Daily trading volume patterns
03	OHLC Plot	Open, High, Low, Close comparison
04	Price Distribution	Histogram + KDE — right-skewed distribution
05	Return Distribution	Histogram + Q-Q plot — heavy tails (leptokurtic)
06	Seasonal Boxplots	Price by year and month
07	Correlation Heatmap	Feature correlations (OHLC ~0.99)
08	Rolling Statistics	30/90-day moving averages + volatility

5.3 Feature Engineering

Adds 3 Bitcoin Whitepaper Protocol Features:

Feature	Source	What It Captures
Block_Reward	Bitcoin protocol	Current mining reward (50→25→12.5→6.25→3.125)
Days_Since_Halving	Halving dates	Days elapsed since last halving event
Halving_Progress	Derived	Fraction through current epoch (0.0 → 1.0)

Why these features? Traditional technical indicators (SMA, RSI) are *reactive* — they lag behind price. Whitepaper features are *deterministic* — known ahead of time from the Bitcoin protocol, anchoring the model to Bitcoin's structural supply-side cycles.

5.4 Leak-Free Scaling

```
split_idx = int(len(data) * 0.8) # 80/20 split
scaler.fit(data[features].iloc[:split_idx]) # Train ONLY
scaled_data = scaler.transform(data[features]) # Apply to all
```

***Why this matters:** If you fit the scaler on the entire dataset, training data 'sees' the statistical properties (min/max) of test data. This is **data leakage** — it inflates metrics and makes them unreliable.*

Final Feature Vector (9 features): Price, Open, High, Low, Vol., Change %, Block_Reward, Days_Since_Halving, Halving_Progress

6. Step 2: Model Training (PyTorch)

6.1 Sequence Generation (Sliding Window)

```
Input: [Day 1, Day 2, ..., Day 60] → 60-day window × 9 features = (60, 9)
Output: [r_1D, r_3D, r_7D] → 3 log return targets
```

Why log returns instead of raw prices?

1. **Stationarity:** Raw prices are non-stationary. Log returns fluctuate around 0.
2. **Scale invariance:** A \$100 move at \$1,000 (10%) ≠ at \$60,000 (0.17%).
3. **Symmetry:** +5% gain and -5% loss are equally sized in log space.

6.2 Train/Test/Validation Split

Partition	Size	Purpose
Training	80% (~3,971 days)	Model learns patterns
■ ■ Train subset	85% of training	Gradient updates
■ ■ Validation subset	15% of training	Early stopping monitor
Test	20% (~993 days)	Final evaluation (never seen)

Critical: `shuffle=False` in `DataLoader` — time series data must maintain temporal order!

6.3 Custom Loss: DirectionalMSELoss

```
loss = MSE(y_pred, y_true) + alpha × mean(ReLU(-y_pred × sign(y_true)))
```

Adds a **directional penalty**: if the model predicts 'up' but actual is 'down', it receives extra penalty ($\alpha=0.15$). Standard MSE only penalizes magnitude error. In trading, getting the direction wrong is much worse!

6.4 Training Configuration

Parameter	Value	Why
Optimizer	Adam	Adaptive learning rate per-parameter
Learning Rate	0.001 / 0.0005	Deeper models need slower learning
Epochs	10	Fast for live viva; pre-saved weights exist
Batch Size	64	Balance gradient noise vs speed
Early Stopping	Patience=3	Stop if val_loss stalls 3 epochs
LR Scheduler	ReduceLROnPlateau	Halve LR when validation plateaus
Gradient Clipping	max_norm=1.0	Prevent exploding gradients in RNNs

6.5 Price Reconstruction

```
predicted_price = anchor_price × exp(predicted_log_return)
```

This is the inverse of the log return formula: $r = \ln(P[t+h] / P[t])$

7. Step 3: Visualization Generation

File: src/step3_viz.py

#	Visualization	What It Shows
09	Loss Curves	Train vs. val loss over epochs — checks for overfitting
10	Actual vs Predicted	Time series overlay of real vs predicted prices
11	Error Distribution	Histogram of errors — should center at 0
12	MAE Bar Charts	Side-by-side MAE comparison across models
13	RMSE Bar Charts	Side-by-side RMSE comparison
14	MAPE Bar Charts	Side-by-side MAPE comparison
15	Grouped Horizon Comparison	How each model degrades across horizons
16	Scatter: Actual vs Predicted	Perfect prediction = 45° line

8. Step 4: Walk-Forward Validation

File: src/step4_wfv.py

Standard train/test split uses a fixed boundary. WFV uses an **expanding window** that simulates real-world deployment:

Fold	Train Period	Test Period	Market Regime
Fold 1	2010 → 2016-07-09	2016-07-09 → 2020-05-11	Halving Epoch 2 (~4 yrs)
Fold 2	2010 → 2020-05-11	2020-05-11 → 2024-03-24	Halving Epoch 3 (~4 yrs)

Why halving-aligned folds? Bitcoin's 4-year halving cycle is its natural market epoch. Each test window covers one complete bull→bear→recovery cycle, ensuring the model is evaluated across a full market cycle — not just a cherry-picked regime.

Key WFV Principles:

- Expanding window: Each fold trains on more data than the last
- No future leakage: Scaler re-fit per fold on training data only
- Fresh model: New model instantiated from scratch per fold
- Realistic evaluation: Tests generalization to unseen market regimes

9. Model Architectures Deep Dive

9.1 1D-CNN

```
Input(60,9) → Transpose → Conv1D(9→64,k=3,causal) → ReLU → Conv1D(64→64) → ReLU  
→ Conv1D(64→32) → ReLU → AvgPool → Linear(32→64) → ReLU → Dropout →  
Linear(64→3)
```

Causal Padding: Only pads on one side, ensuring each output depends only on past/present inputs — not future values.

9.2 RNN (Simple Recurrent)

```
Input(60,9) → RNN(9→64) → Dropout → RNN(64→32) → Last timestep → Dropout →  
Linear(32→32) → ReLU → Linear(32→3)
```

Vanishing Gradients: In long sequences (60 steps), gradients shrink exponentially during backpropagation through time. RNNs struggle with long-range dependencies.

9.3 LSTM (Long Short-Term Memory)

```
Input(60,9) → LSTM(9→128) → Dropout → LSTM(128→64) → Dropout → LSTM(64→32) →  
Last step → Dropout → Linear(32→64) → ReLU → Linear(64→3)
```

Gates solve vanishing gradients: Forget gate (discard old info), Input gate (store new info), Output gate (what to output). These provide a gradient 'highway'.

9.4 Transformer

```
Input(60,9) → Linear(9→256) → 2× TransformerEncoder(d=256, 4 heads, ff=128) →  
AvgPool → Dropout → Linear(256→64) → ReLU → Dropout → Linear(64→3)
```

Self-Attention: Each timestep attends to ALL others simultaneously (not sequentially). Captures long-range dependencies but processes 60 tokens = $O(60^2)$ complexity.

9.5 PatchTST (Best Model)

```
Input(60,9) → RevIN(normalize) → Unfold into 5 patches of size 12 →  
Flatten(12×9=108) → Linear(108→128) → +PosEmbed → 3× TransformerEncoder(d=128, 4  
heads, ff=256, GELU) → Flatten(5×128=640) → Linear(640→64) → ReLU → Dropout →  
Linear(64→3)
```

Why PatchTST wins:

1. **Patching:** Reduces 60 tokens → 5 tokens. Attention complexity drops from $O(3600)$ to $O(25)$ = 96% reduction.
2. **RevIN:** Normalizes per-sample mean/std before encoding, reverses after. Handles distribution shift.
3. **GELU:** Smoother than ReLU, allows small negative values to pass through.

10. Dashboard Pages (10 Pages)

#	Page	What It Shows
0	Overview (Landing)	Project summary, KPI cards, model descriptions
1	Exploratory Analysis	Price trends, outlier box plots, return distribution
2	Seasonality Analysis	Intra-month return heatmap (Q1-Q4), win rates
3	Model Performance	Leaderboard table, dynamic bar charts (all 5 models)
4	Backtest (WFOV)	Walk-forward validation stats, stability checks
5	Diagnostics	Actual vs predicted selectors, loss curves
6	Macro & Halving	Liquidity rotation, halving schedule
7	Live Prediction	Real-time BTC (Yahoo Finance) → live PyTorch inference
8	Forward Testing	Out-of-sample evaluation (Mar 2024 → present)
9	Explainable AI	SHAP-based feature importance
10	Trading Simulator	Simulated trading strategy & ROI

11. Key Data Science Concepts for Viva

Why Log Returns?

Raw prices are non-stationary (violates ML assumptions). Log returns are approximately stationary, additive over time, and symmetric. Formula: $r = \ln(P[t+h] / P[t])$

Data Leakage Prevention

Three layers: (1) Chronological split (no shuffle), (2) Scaler fit on train only, (3) Walk-forward validation with expanding window.

The R^2 Paradox

Price-level $R^2 \approx 0.99$ is misleading — yesterday's price predicts ~99% of today's level. Return-level $R^2 \approx 1-3\%$ is the true measure. In quant finance, 2% is considered highly successful.

Multi-Output Architecture

All 5 models produce 3 outputs simultaneously: $\text{Linear}(\text{hidden}, 3) \rightarrow [r_{1D}, r_{3D}, r_{7D}]$. The shared encoder learns representations useful across all horizons.

MinMaxScaler vs StandardScaler

MinMaxScaler scales to $[0,1]$ — works best with bounded NN activations. StandardScaler (mean=0, std=1) is better with outliers, but BTC outliers are meaningful.

12. Results Summary

Best Results per Horizon:

Horizon	Best Model	MAE (USD)	RMSE (USD)	MAPE (%)
1-Day	PatchTST	\$732.23	\$1,138.47	2.06%
3-Day	PatchTST	\$1,347.56	\$1,962.13	3.85%
7-Day	PatchTST	\$2,097.01	\$3,067.21	5.93%

All Model Rankings (by MAPE at 1D Horizon):

Rank	Model	1D MAPE	3D MAPE	7D MAPE
1	PatchTST	2.06%	3.85%	5.93%
2	LSTM	2.13%	4.07%	6.36%
3	Transformer	2.76%	4.15%	6.41%
4	RNN	2.85%	4.25%	6.23%
5	1D-CNN	5.48%	5.33%	7.16%

13. Common Viva Questions & Answers

Q1: Why PyTorch over TensorFlow?

PyTorch provides dynamic computation graphs (eager mode), making debugging easier. It's the dominant framework in research papers (including PatchTST). Custom architectures like RevIN are simpler to implement.

Q2: Why 60-day sequence length?

~2 months captures short-term trends, weekly patterns, and monthly cycles. Too short (7 days) misses medium-term patterns. Too long (365 days) introduces noise and increases Transformer complexity quadratically.

Q3: How does the model handle the 2024 halving?

Block_Reward drops from 6.25 to 3.125, and Halving_Progress resets near 0. The model has learned from 3 prior halvings. However, only 4 cycles exist total (Data Starvation bottleneck).

Q4: What is early stopping?

Monitors validation loss and stops training when it stops improving (patience=3 epochs). Prevents overfitting — memorizing training noise instead of learning generalizable patterns.

Q5: Why is 1D-CNN the weakest?

CNNs only capture local patterns within the kernel window (3 timesteps). Cannot model long-range dependencies spanning hundreds of timesteps. RNNs, LSTMs, and Transformers all have long-range memory.

Q6: What is gradient clipping?

`clip_grad_norm_(params, 1.0)` limits gradient magnitude to 1.0. Without this, RNNs suffer from exploding gradients where error signals grow exponentially during backpropagation through time.

Q7: Explain DirectionalMSELoss.

Combines MSE (magnitude error) with a directional penalty. If model predicts 'up' but actual is 'down', extra loss is added ($\alpha=0.15$). Crucial for trading where direction matters more than magnitude.

Q8: What makes WFV better than simple train/test?

A single split only tests one market regime. WFV tests across multiple regimes (Epochs 2 and 3). Gives confidence the model generalizes across bull and bear markets.